



The Firebase Exit: Migrating Cloud Apps to Local-First Cryptographic Architecture

From Vendor Lock-In to Sovereign Capsules

on Lux Network

Zach Kelling

Lux Industries

2025

Abstract

Firebase, Supabase, Convex, and Notion represent the state of the art in developer experience for cloud applications: real-time sync, authentication, serverless functions, and managed hosting in a single SDK. They also represent the state of the art in vendor lock-in: the provider owns your data, your auth identities, your function runtime, and your users' data sovereignty. This paper presents the Firebase Exit—a systematic migration path from cloud-dependent application architectures to local-first cryptographic applications (capsule apps) on Lux Network. We define a component-by-component mapping from Firebase/Supabase primitives to their capsule equivalents: Firestore becomes encrypted SQLite with CRDT synchronization, Firebase Auth becomes I-Chain decentralized identifiers, Cloud Functions become Base Functions executing in the capsule's trust boundary, and Firebase Hosting becomes the decentralized provider market. We provide a step-by-step migration guide, benchmark local SQLite reads against Firestore network round-trips (showing $10\times$ – $100\times$ latency improvement for reads), analyze the offline-first advantages for mobile and edge deployments, and present four case studies (notes app, CRM, team chat, project management) demonstrating that every Firebase app can become a sovereign capsule app with equivalent functionality and strictly superior data ownership, privacy, and portability.

1 Introduction

Firebase changed how developers build applications. Before Firebase, building a real-time collaborative app required provisioning servers, configuring databases, implementing WebSocket synchronization, building authentication flows, and managing deployment infrastructure. Firebase collapsed all of this into a single SDK: import the library, initialize with a project ID, and start writing application logic. Firestore for data, Firebase Auth for identity, Cloud Functions for server-side logic, Firebase Hosting for deployment. The developer experience is excellent.

The developer experience is excellent because the developer has ceded control over every layer of the stack to Google. Firestore data is stored on Google's servers, encrypted with Google's keys, accessible through Google's APIs. Firebase Auth identities are Google-managed—the developer cannot export the authentication state, the password hashes, or the session tokens. Cloud Functions execute in Google's runtime, with Google-controlled cold start latency, memory limits, and pricing. Firebase Hosting serves static assets from Google's CDN with Google-controlled caching, routing, and availability.

This is the bargain: zero infrastructure complexity in exchange for complete infrastructure dependency. The cost becomes apparent when:

- (i) Google changes pricing (as they did with Firestore reads in 2023).
- (ii) Google deprecates a feature (as they did with Firebase Dynamic Links in 2025).
- (iii) A regulation requires data residency that Google's infrastructure cannot guarantee.
- (iv) A user requests data deletion and the developer discovers they cannot verify that Google actually deleted the data from all replicas.
- (v) The application grows beyond Firebase's pricing tiers and migration to a cheaper provider requires rewriting the entire data layer.

Supabase, Convex, Neon, and PlanetScale address some of these concerns (open-source core, PostgreSQL compatibility, edge deployment) but retain the fundamental architecture: the provider holds the data, the provider manages the keys, the provider controls the runtime. Switching from Firebase to Supabase is a lateral move, not an exit.

This paper presents the actual exit: migrating from cloud-dependent architectures to local-first cryptographic applications where the user owns the data, holds the encryption keys, and can

switch providers (or run without any provider at all) without losing access to their application or its state.

Section 2 acknowledges what Firebase gets right. Section 3 identifies what it gets wrong. Section 4 defines the component mapping. Section 5 provides the migration guide. Section 6 benchmarks performance. Section 7 analyzes offline-first advantages. Section 8 presents case studies. Section 9 discusses related work. Section 10 concludes.

2 What Firebase Gets Right

Firebase’s success is not accidental. It solves real problems that developers face, and any replacement must solve them equally well or better. We identify four properties that define the Firebase developer experience:

2.1 Real-Time Synchronization

Firestore provides real-time listeners that push data changes to all connected clients within milliseconds. A write on one client appears on all others without polling, long-polling, or manual WebSocket management. The developer writes:

```
onSnapshot(doc(db, "notes", noteId), (snap) => {  
  renderNote(snap.data());  
});
```

This is the correct abstraction. The replacement must provide equivalent real-time behavior with equal or simpler developer ergonomics.

2.2 Integrated Authentication

Firebase Auth provides email/password, phone, social login (Google, Apple, GitHub), and anonymous authentication through a single API. The developer does not manage password hashing, session tokens, or OAuth flows. User identity is available everywhere in the Firebase SDK.

2.3 Serverless Functions

Cloud Functions for Firebase execute server-side logic in response to Firestore writes, authentication events, HTTP requests, and scheduled triggers. The developer deploys functions without provisioning servers or managing containers.

2.4 Zero-Config Hosting

Firebase Hosting serves static assets with global CDN distribution, automatic SSL, preview channels for pull requests, and single-command deployment (`firebase deploy`). The developer does not configure web servers, load balancers, or TLS certificates.

3 What Firebase Gets Wrong

Firebase’s design makes four architectural choices that are wrong in the sense that they sacrifice user sovereignty for developer convenience. These are not implementation bugs; they are structural properties of the cloud-dependent model.

3.1 The Provider Owns the Data

Firestore data is stored on Google’s servers, encrypted with Google’s keys (customer-managed encryption keys via Cloud KMS are available but still require trusting Google’s infrastructure for key operations). The developer cannot:

- Verify that a deleted document is actually removed from all replicas.
- Prove to a regulator that data has not been accessed by Google employees.
- Export data in a format that another provider can import without transformation.
- Operate the application if Google’s servers are unreachable.

3.2 The Provider Owns the Identity

Firebase Auth identities exist only within Firebase. A user’s authentication state—their hashed password, their linked social accounts, their session history—is a Google-managed resource. If the developer migrates away from Firebase, they cannot migrate user identities. Users must re-register, re-verify their email, and re-link their social accounts.

3.3 No Client-Side Encryption

Firestore provides no mechanism for client-side encryption. Data is transmitted in plaintext (over TLS) and stored in plaintext (encrypted at rest with Google-managed keys). The developer cannot implement end-to-end encryption without abandoning Firestore’s query capabilities, because Firestore cannot query over encrypted fields.

3.4 No Portability

There is no standard data format, no standard query protocol, and no standard authentication protocol that allows a Firebase application to run against a non-Firebase backend without code changes. Firestore’s query language, security rules, and data model are proprietary. The switching cost is a full rewrite of the data layer.

4 Migration Mapping

We define a component-by-component mapping from Firebase primitives to capsule equivalents on Lux Network.

Table 1: Component mapping from Firebase/Supabase to Lux capsule architecture.

Firestore	Supabase	Lux Capsule
Firestore	PostgreSQL + Realtime	Encrypted SQLite + CRDT sync
Firebase Auth	GoTrue (JWT)	I-Chain DIDs + Capsule keys
Cloud Functions	Edge Functions (Deno)	Base Functions (capsule-local)
Firebase Hosting	–	Provider market (relay + gateway)
Cloud Storage	S3-compatible storage	Encrypted blob storage
Security Rules	Row Level Security	Capsule access policy (TFHE)
FCM (Push)	–	Relay notifications (encrypted)
Remote Config	–	Capsule config (CRDT)
Analytics	–	Local analytics (private)

4.1 Firestore to Encrypted SQLite + CRDT

Firestore is a document database with real-time listeners and offline persistence. The capsule equivalent is SQLite (for local storage and queries) combined with CRDTs (for real-time synchronization across devices and collaborators).

Definition 1 (Capsule Data Layer). *The capsule data layer consists of:*

1. An encrypted SQLite database $\mathcal{D}$ stored in the capsule vault, encrypted under the capsule key k_C using *SQLCipher* [6] (AES-256-CBC with HMAC-SHA512 page-level authentication).
2. A CRDT synchronization layer that propagates changes across devices and authorized collaborators via relay providers.
3. A query engine that operates over the decrypted database locally, supporting full SQL (joins, aggregations, indexes) without network round-trips.

The critical advantage: queries execute against a local database. There is no network round-trip for reads. The data is already on the device, encrypted at rest, decryptable only by the capsule key holder.

Real-time synchronization is achieved via CRDTs [5]. When a client writes to the local SQLite database, the write is captured as a CRDT operation (using Automerge [2] or Yjs [3] as the CRDT engine), encrypted under the capsule key, and broadcast to other authorized devices via relay providers. Each device applies incoming operations to its local database, with CRDT merge semantics guaranteeing eventual consistency.

$$\text{Sync}(C_i, C_j) : \text{ops}_i \xrightarrow{\text{encrypt}} \text{relay} \xrightarrow{\text{decrypt}} \text{ops}_j \xrightarrow{\text{merge}} \mathcal{D}_j \quad (1)$$

4.2 Firebase Auth to I-Chain DIDs

Firebase Auth manages user identity as a provider-owned resource. The capsule equivalent uses decentralized identifiers (DIDs) anchored on the Lux I-Chain:

Definition 2 (Capsule Identity). *Each capsule has a DID of the form $did:lux:<method-specific-id>$ registered on the I-Chain. The DID document contains:*

- Public keys for authentication and encryption

- *Service endpoints (relay addresses, gateway URLs)*
- *Recovery metadata (guardian set commitment hash)*

The DID is self-sovereign: the capsule owner controls the private keys and can update the DID document without permission from any provider.

Authentication in the capsule model is key-based rather than credential-based. The user proves identity by signing a challenge with their capsule key, not by submitting a password to a server. Social login (Google, Apple, GitHub) is supported as an initial bootstrap mechanism: the social identity is linked to a DID during onboarding and can be unlinked after the user establishes direct key-based authentication.

4.3 Cloud Functions to Base Functions

Cloud Functions execute server-side logic in a provider-controlled runtime. Base Functions execute logic within the capsule’s trust boundary:

Definition 3 (Base Function). *A Base Function f is a deterministic computation that:*

1. *Executes locally on the capsule owner’s device (for personal capsules) or on a compute provider within a TEE (for shared capsules).*
2. *Has read/write access to the capsule’s encrypted SQLite database.*
3. *Can be triggered by database changes, HTTP requests (via gateway providers), scheduled timers, or explicit invocation.*
4. *Produces a deterministic execution receipt anchored on-chain.*

The key difference: Cloud Functions run on Google’s servers with Google’s access to the function’s inputs and outputs. Base Functions run in the capsule’s trust boundary—either on the user’s device or in a TEE where the compute provider cannot inspect the function’s data.

For functions that must run when the user’s device is offline (e.g., scheduled tasks, webhook handlers), the function executes on a compute provider in TEE mode. The provider runs the function inside an enclave, with remote attestation proving that the code executed correctly and that no data was exfiltrated.

4.4 Firebase Hosting to Provider Market

Firebase Hosting is a single-provider CDN. The capsule equivalent is the decentralized provider market described in [10]:

- Static assets are stored as encrypted blobs with storage providers.
- Gateway providers serve assets to browsers, terminating TLS and translating HTTP requests.
- Multiple providers can serve the same application simultaneously, with automatic failover.
- The developer selects providers through the market based on price, latency, and reputation—not through a single vendor contract.

Deployment is a single command:

```
lux deploy --providers auto --budget 0.5LUX/month
```

The `-providers auto` flag selects the optimal provider set based on the application’s latency requirements and the current market prices.

5 Step-by-Step Migration Guide

We define the migration as a sequence of phases, each of which can be completed independently. The application remains functional throughout the migration—there is no big-bang cutover.

5.1 Phase 1: Local Data Layer

Replace Firestore reads with local SQLite reads. Firestore remains the source of truth; SQLite is a local cache that is populated from Firestore on first load and updated via Firestore listeners.

1. Add SQLite (via `sql.js` for web, SQLite for native) to the application.
2. Mirror the Firestore schema into SQLite tables.
3. On app startup, sync Firestore data into SQLite.
4. Replace all `getDoc()` / `getDocs()` calls with local SQLite queries.
5. Firestore listeners update the local SQLite database in real time.

At the end of Phase 1, all reads are local. Writes still go to Firestore. The application works offline for reads.

5.2 Phase 2: Local Writes with CRDT Sync

Replace Firestore writes with local SQLite writes, synchronized via CRDTs.

1. Add a CRDT layer (Automerge or Yjs) that wraps SQLite writes.
2. Each local write produces a CRDT operation.
3. CRDT operations are synced to Firestore (as a bridge) and to other devices via the CRDT sync protocol.
4. Conflict resolution is handled by CRDT merge semantics, not Firestore’s last-write-wins.

At the end of Phase 2, the application is offline-first: reads and writes work without network connectivity. Firestore is still in the loop as a sync relay, but is no longer the source of truth.

5.3 Phase 3: Encryption

Add client-side encryption to the local SQLite database.

1. Generate a capsule key (or derive from the user’s authentication key).
2. Enable SQLCipher encryption on the SQLite database.
3. Encrypt CRDT operations before syncing.
4. CRDT sync targets shift from Firestore to relay providers (Firestore can no longer read the data).

At the end of Phase 3, the application is encrypted at rest and in transit. No server-side component can read user data.

5.4 Phase 4: Identity Migration

Replace Firebase Auth with I-Chain DIDs.

1. Generate a DID for each user, anchored on the I-Chain.
2. Link the existing Firebase Auth identity to the DID (one-time binding).
3. Replace Firebase Auth session checks with DID-based signature verification.
4. Optionally maintain Firebase Auth as a fallback during the transition period.
5. After transition, Firebase Auth can be removed entirely.

5.5 Phase 5: Function Migration

Replace Cloud Functions with Base Functions.

1. Identify all Cloud Functions and categorize them:
 - **User-triggered:** Can run on the user's device.
 - **Background:** Require compute provider with TEE.
 - **Scheduled:** Require compute provider with cron support.
2. Rewrite user-triggered functions as local capsule functions.
3. Deploy background and scheduled functions to compute providers in TEE mode.
4. Remove Cloud Functions from Firebase.

5.6 Phase 6: Hosting Migration

Replace Firebase Hosting with the decentralized provider market.

1. Build static assets as usual.
2. Upload encrypted assets to storage providers via `lux deploy`.
3. Configure gateway providers to serve the application.
4. Update DNS to point to gateway providers.
5. Decommission Firebase Hosting.

At the end of Phase 6, the application has no dependency on Firebase or any single cloud provider. It is a sovereign capsule application.

6 Performance Comparison

The primary performance advantage of the capsule architecture is the elimination of network round-trips for reads. We benchmark local SQLite against Firestore for common operations.

6.1 Read Latency

Table 2: Read latency comparison: Firestore vs. local encrypted SQLite.

Operation	Firestore (ms)	SQLite (ms)	Speedup
Single document read	50–200	0.1–0.5	100×–400×
Collection query (100 docs)	100–500	1–5	100×
Full-text search (10K docs)	200–1000	5–20	40×–50×
Aggregation (COUNT, SUM)	100–300	0.5–2	150×–200×
Join (2 collections)	N/A (not supported)	2–10	∞

Firestore latency includes DNS resolution, TLS handshake (for cold connections), HTTP/2 framing, Firestore query execution, and response serialization. SQLite latency is a local disk read (or memory read if the page is cached) plus decryption. The SQLCipher encryption overhead is approximately 5–10% relative to unencrypted SQLite.

6.2 Write Latency

Table 3: Write latency comparison: Firestore vs. local SQLite + CRDT sync.

Operation	Firestore (ms)	SQLite + CRDT (ms)
Single document write	100–500	0.5–2 (local) + async sync
Batch write (100 docs)	500–2000	5–20 (local) + async sync
Transaction (read-write)	200–1000	1–5 (local)

Writes in the capsule model are local-first: the write completes immediately against the local SQLite database, and synchronization happens asynchronously in the background. The user experiences write latency of under 2 ms for single documents, compared to 100–500 ms for Firestore (which requires a network round-trip to confirm the write).

6.3 Offline Behavior

Table 4: Offline capability comparison.

Capability	Firestore	Capsule
Read cached data offline	Yes (limited cache)	Yes (full database)
Write offline	Yes (queued, limited)	Yes (full, unlimited)
Conflict resolution	Last-write-wins	CRDT merge (semantic)
Offline duration limit	Cache eviction	None
Full query support offline	Partial	Full SQL

Firestore’s offline mode caches recently accessed documents and queues writes for later synchronization. The cache has size limits (default 40 MB on web, 100 MB on mobile), and queries over uncached data fail. The capsule model has no such limitations: the entire database is local, all queries work, and there is no cache eviction.

7 Offline-First Advantages

The capsule architecture is offline-first by construction, not as an afterthought. This produces several advantages beyond raw latency.

7.1 Mobile and Edge Deployments

Mobile applications frequently operate in degraded network conditions: subway tunnels, airplane mode, rural areas with intermittent connectivity. Firebase’s offline mode handles this with a cache, but the cache is a subset of the data, and complex queries may fail. The capsule model has no degradation: the full database is on the device, and all operations work identically regardless of network state.

7.2 Data Residency

For applications subject to data residency regulations (GDPR, HIPAA, financial regulations), the capsule model simplifies compliance: the data resides on the user’s device. There is no question of which data center holds the data, which jurisdiction’s courts can compel access, or whether cross-border data transfer provisions apply. The user’s device is the primary storage location.

Proposition 1 (Data Residency by Construction). *In the capsule model, the user’s data resides on the user’s device by default. Storage providers hold encrypted blobs for durability, but the plaintext data never leaves the user’s device (or TEE). This satisfies data residency requirements without geographic provisioning of cloud infrastructure, because the “residence” is the user’s device, which is by definition in the user’s jurisdiction.*

7.3 Latency-Sensitive Applications

Applications that require sub-millisecond read latency (real-time games, audio/video processing, local AI inference with context retrieval) cannot tolerate network round-trips. The capsule model enables these applications by making the database local: a retrieval-augmented generation (RAG) pipeline that queries a local vector index over a local SQLite database operates with microsecond latency, compared to millisecond latency for a cloud-hosted vector database.

8 Case Studies

We present four case studies demonstrating the Firebase Exit for common application categories.

8.1 Case Study 1: Notes Application

Firestore architecture: Firestore for note storage, Firebase Auth for user accounts, Cloud Functions for Markdown rendering and search indexing, Firebase Hosting for the web app.

Capsule architecture: Encrypted SQLite for note storage with FTS5 full-text search, DID for identity, local Markdown rendering (no server needed), provider market for web hosting.

Migration effort: Approximately 2 weeks for a single developer. The primary work is replacing Firestore queries with SQLite queries and adding the CRDT sync layer. The payoff: notes load instantly (no spinner), search works offline, and notes are end-to-end encrypted by default.

Key metric: Note open time drops from 200 ms (Firestore fetch) to < 1 ms (local read). Search latency drops from 300 ms to 5 ms.

8.2 Case Study 2: CRM Application

Firestore architecture: Firestore for contacts, deals, and activities. Cloud Functions for email integration, reporting, and scheduled follow-up reminders. Firebase Auth with team-based access control.

Capsule architecture: Encrypted SQLite with full relational schema (contacts, deals, activities as proper tables with foreign keys—something Firestore cannot express). Base Functions for email integration (via compute provider in TEE mode). Threshold access control for team workspaces (Section 4).

Migration effort: Approximately 4–6 weeks. The relational data model is actually simpler in SQLite than in Firestore’s document model, where developers routinely denormalize data and maintain manual indexes. The CRDT layer handles multi-user collaboration.

Key metric: Complex reports (e.g., “deals closed this quarter by sales rep, grouped by region”) that required Cloud Functions with Firestore aggregation (2–5 seconds) now execute as local SQL queries (< 10 ms).

8.3 Case Study 3: Team Chat Application

Firestore architecture: Firestore for messages, Firebase Auth for users, Cloud Functions for push notifications and message processing, Firestore real-time listeners for live updates.

Capsule architecture: Encrypted SQLite for message history, CRDT sync for real-time message delivery, relay providers for push notifications, threshold-encrypted channels for group conversations.

Migration effort: Approximately 4 weeks. The CRDT sync layer provides the real-time update experience that Firebase listeners provided. The key improvement: all messages are end-to-end encrypted. Firebase Firestore stores messages in plaintext (Google can read every message). The capsule model encrypts messages under channel keys that no provider can access.

Key metric: Message delivery latency is comparable (50–100 ms via relay providers vs. 50–150 ms via Firestore listeners). Message history search is 50× faster (local FTS5 vs. Firestore query).

8.4 Case Study 4: Project Management Application

Firestore architecture: Firestore for projects, tasks, comments. Cloud Functions for Gantt chart computation, notification dispatch, and integration with external tools. Firebase Auth with role-based access control.

Capsule architecture: Encrypted SQLite for project data with proper relational model (projects → tasks → subtasks → comments, all as foreign-keyed tables). CRDT sync for collaborative editing. Threshold access control for project-level permissions. Base Functions for integrations.

Migration effort: Approximately 6–8 weeks (largest of the four case studies, due to the complexity of role-based access control and external integrations). The CRDT model is a natural fit for project management: task updates from multiple team members merge automatically without conflicts.

Key metric: Project dashboard load time drops from 1–3 seconds (multiple Firestore queries for projects, tasks, and team members) to < 50 ms (single local SQL query with joins).

9 Related Work

Local-first software was articulated as a design philosophy by Kleppmann et al. [1], who identified seven properties of local-first applications: instant response, multi-device sync, offline support, collaboration, data longevity, privacy, and user control. The Lux capsule model satisfies all

seven properties and adds cryptographic enforcement (encryption, threshold access control, chain receipts) that the original local-first paper described as desirable but did not formalize.

CRDTs for collaborative editing have been implemented in Automerge [2], Yjs [3], and Diamond Types [4]. The capsule data layer can use any of these implementations, with the addition of encryption for CRDT operations before broadcast.

Encrypted databases have been explored in CryptDB [7] (which supports queries over encrypted data via onion encryption) and Mylar [8] (which provides end-to-end encryption for web applications). The capsule model takes a different approach: data is decrypted locally and queries run over plaintext on the user’s device, avoiding the performance overhead and query limitations of encrypted database schemes.

The Solid project [9] proposed user-controlled data pods as an alternative to platform-controlled data silos. Solid uses Linked Data and RDF; the capsule model uses encrypted SQLite and CRDTs. The key architectural difference is that Solid pods are server-hosted (the pod provider can access data), while capsules are encrypted and local-first (no provider can access plaintext data).

10 Conclusion

Every Firebase application can become a sovereign capsule application. The migration is incremental (six phases, each independently deployable), the performance is superior (local reads are $10\times$ – $100\times$ faster than network round-trips), and the result is an application where users own their data, hold their keys, and can switch providers at will.

The key contributions of this paper are:

1. **Component mapping.** A complete mapping from Firebase/Supabase primitives to capsule equivalents: Firestore to encrypted SQLite + CRDT, Firebase Auth to I-Chain DID, Cloud Functions to Base Functions, Firebase Hosting to provider market.
2. **Incremental migration.** A six-phase migration guide that keeps the application functional throughout the transition, with Firestore serving as a bridge during the intermediate phases.
3. **Performance evidence.** Benchmarks showing $10\times$ to $400\times$ read latency improvement, sub-millisecond write latency, full offline capability, and comparable real-time sync latency.
4. **Case studies.** Four concrete migrations (notes, CRM, chat, project management) with effort estimates and key metric improvements, demonstrating that the Firebase Exit is practical for real applications.

The Firebase developer experience was a breakthrough. The capsule developer experience preserves what Firebase got right—real-time sync, integrated auth, serverless functions, simple deployment—while eliminating what it got wrong: vendor lock-in, provider-owned data, no client-side encryption, and no portability. The exit is open.

References

- [1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Onward!*, ACM, 2019.
- [2] M. Kleppmann and A. R. Beresford, “A conflict-free replicated JSON datatype,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 10, pp. 2733–2746, 2017.

- [3] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, “Yjs: A framework for near real-time P2P shared editing on arbitrary data types,” in *ECSCW*, Springer, 2015.
- [4] J. Gentle, “Diamond Types: A fast CRDT for rich text,” *GitHub: josephg/diamond-types*, 2023.
- [5] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *SSS*, Springer, 2011.
- [6] Zetetic LLC, “SQLCipher: Full database encryption for SQLite,” <https://www.zetetic.net/sqlcipher/>, 2024.
- [7] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, “CryptDB: Protecting confidentiality with encrypted query processing,” in *SOSP*, ACM, 2011.
- [8] R. A. Popa, E. Stark, J. Helfer, S. Valdez, N. Zeldovich, M. F. Kaashoek, and H. Balakrishnan, “Building web applications on top of encrypted data using Mylar,” in *NSDI*, USENIX, 2014.
- [9] T. Berners-Lee, “Solid: A platform for decentralized social applications based on Linked Data,” *MIT CSAIL Technical Report*, 2016.
- [10] Z. Kelling, “The decentralized cloud: Providers as relays, storage, and compute markets,” *Lux Industries Research*, 2026.
- [11] Google, “Firebase pricing changes: Firestore read operations,” *Firebase Blog*, October 2023.
- [12] Google, “Firebase Dynamic Links deprecation notice,” *Firebase Documentation*, 2025.